

CO2-AT3

1. Online Food Delivery Data Analysis

Libraries Used: Pandas, Plotly Express

A. Using Pandas to Read, Clean, and Filter Order Data

Pandas is used to create and manage the food delivery dataset containing Restaurant Name, Delivery Time, Price, Customer Rating, and Order Details. It organizes the data into a DataFrame, making it easy to analyze. Missing delivery times are replaced with the average value using `fillna()`, duplicate records are removed using `drop_duplicates()`, and invalid prices are filtered out. Functions like `groupby()` and `value_counts()` are used to calculate restaurant-wise orders and average delivery time.

Application:

- Organize food delivery records.
- Remove duplicate and incorrect data.
- Count restaurant-wise orders.
- Calculate average delivery time.
- Filter highly rated customer orders.

B. Data Preprocessing

Data preprocessing improves the quality of the dataset before analysis.

- Missing delivery times are replaced with the average delivery time.
- Duplicate records are removed.
- Invalid prices are eliminated.
- Customer ratings are filtered to display only highly rated orders.

Benefits:

- Improves data quality.
- Removes inconsistencies.
- Produces accurate analysis.
- Helps in better business decision-making.

C. Visualization using Plotly Express

Plotly Express is used to generate interactive charts.

- Bar Chart: Displays the number of orders received by each restaurant.
- Bar Chart: Shows the average delivery time of each restaurant.
- Histogram: Displays the distribution of customer ratings.

These visualizations help identify popular restaurants, compare delivery performance, and analyze customer satisfaction.

CODE:

```
import pandas as pd
import plotly.express as px

order_data = {
    "Order_ID": [101, 102, 103, 104, 105, 106],
    "Restaurant": ["KFC", "Dominos", "KFC", "Pizza Hut", "Dominos", "KFC"],
    "Delivery_Time": [30, 45, None, 25, 60, 45],
    "Price": [350, 450, 250, -200, 500, 450],
    "Customer_Rating": [5, 4, 5, 3, 2, 4]
}

df = pd.DataFrame(order_data)

print("Original Data")
print(df)

df["Delivery_Time"] = df["Delivery_Time"].fillna(df["Delivery_Time"].mean())
df = df.drop_duplicates()
df = df[df["Price"] > 0]

print("\nCleaned Data")
print(df)

high_rating = df[df["Customer_Rating"] >= 4]

print("\nOrders with Rating >= 4")
print(high_rating)

top_restaurants = df["Restaurant"].value_counts()

print("\nTop Restaurants")
print(top_restaurants)

delay = df.groupby("Restaurant")["Delivery_Time"].mean().reset_index()

print("\nAverage Delivery Time")
print(delay)
```

```

fig1 = px.bar(
    x=top_restaurants.index,
    y=top_restaurants.values,
    labels={"x": "Restaurant", "y": "Orders"},
    title="Top Restaurants by Orders"
)

fig2 = px.bar(
    delay,
    x="Restaurant",
    y="Delivery_Time",
    title="Average Delivery Time"
)

fig3 = px.histogram(
    df,
    x="Customer_Rating",
    title="Customer Ratings Distribution"
)

fig1.show()
fig2.show()
fig3.show()

```

OUTPUT:

Original Data

	Order_ID	Restaurant	Delivery_Time	Price	Customer_Rating
0	101	KFC	30.0	350	5
1	102	Dominos	45.0	450	4
2	103	KFC	NaN	250	5
3	104	Pizza Hut	25.0	-200	3
4	105	Dominos	60.0	500	2
5	106	KFC	45.0	450	4

Cleaned Data

	Order_ID	Restaurant	Delivery_Time	Price	Customer_Rating
0	101	KFC	30.0	350	5
1	102	Dominos	45.0	450	4
2	103	KFC	41.0	250	5
4	105	Dominos	60.0	500	2
5	106	KFC	45.0	450	4

Orders with Rating >= 4

	Order_ID	Restaurant	Delivery_Time	Price	Customer_Rating
0	101	KFC	30.0	350	5
1	102	Dominos	45.0	450	4
2	103	KFC	41.0	250	5
5	106	KFC	45.0	450	4

Top Restaurants

Restaurant

KFC 3

Dominos 2

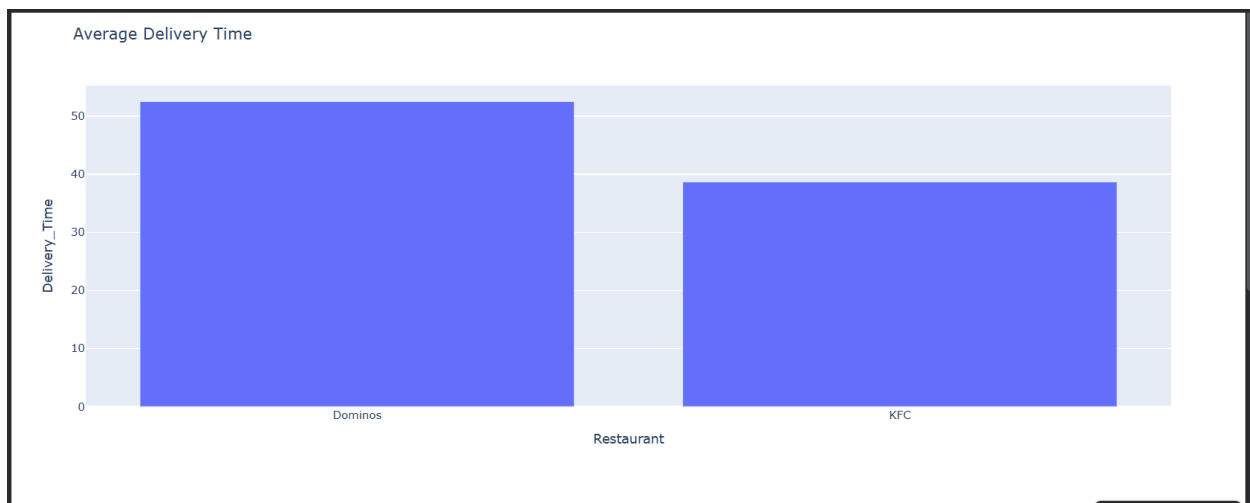
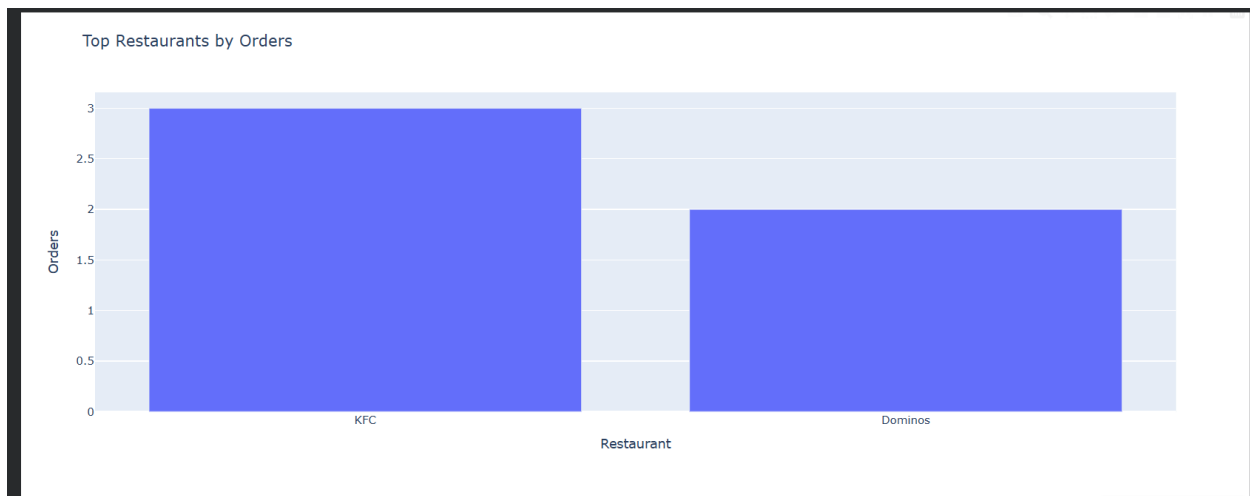
Name: count, dtype: int64

Average Delivery Time

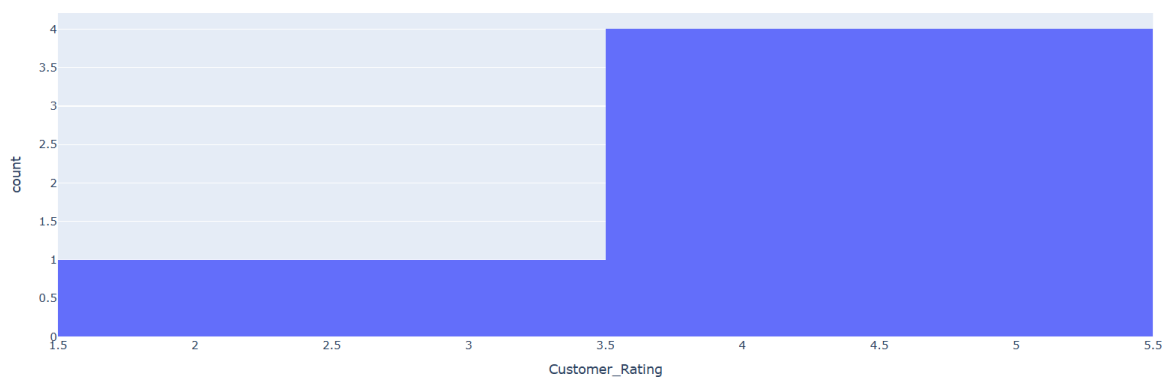
Restaurant Delivery_Time

0 Dominos 52.500000

1 KFC 38.666667



Customer Ratings Distribution



2. Bank Loan Approval Analysis

Libraries Used: Pandas, Scikit-learn

A. Using Pandas DataFrame Operations

Pandas is used to organize customer loan information in a DataFrame containing Income, Credit Score, Loan Amount, and Loan Approval Status. It stores and displays customer details, making the dataset easy to understand and process before applying machine learning.

Application:

- Organize customer loan records.
- Display applicant information.
- Select important loan features.
- Prepare data for prediction.

B. Data Preprocessing

Scikit-learn preprocessing techniques prepare the dataset for machine learning.

- Loan approval values (Yes/No) are converted into numerical values using LabelEncoder.
- The dataset is divided into input features (Income, Credit Score, Loan Amount) and the target value (Approval Status).

These preprocessing steps make the data suitable for machine learning.

Benefits:

- Converts categorical data into numerical form.
- Prepares the dataset for prediction.
- Improves model performance.

C. Loan Prediction using Scikit-learn

Scikit-learn is used to build a Decision Tree Classifier.

- The model is trained using historical loan data.
- It learns the relationship between customer details and loan approval.
- The trained model predicts whether a new applicant's loan will be Approved or Rejected.

This helps banks automate the loan approval process and make faster decisions

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
```

```

data = {
    "Income": [50000, 30000, 70000, 40000, 60000],
    "CreditScore": [750, 600, 800, 650, 720],
    "LoanAmount": [200000, 150000, 250000, 180000, 220000],
    "Approval": ["Yes", "No", "Yes", "No", "Yes"]
}

df = pd.DataFrame(data)

print("Original Data")
print(df)

le = LabelEncoder()
df["Approval"] = le.fit_transform(df["Approval"])

X = df[["Income", "CreditScore", "LoanAmount"]]
y = df["Approval"]

model = DecisionTreeClassifier()
model.fit(X, y)

prediction = model.predict([[55000, 700, 200000]])

result = le.inverse_transform(prediction)

print("\nPrediction for New Applicant:")
print("Loan Approval:", result[0])

```

OUTPUT:

Original Data

	Income	CreditScore	LoanAmount	Approval
0	50000	750	200000	Yes
1	30000	600	150000	No
2	70000	800	250000	Yes
3	40000	650	180000	No
4	60000	720	220000	Yes

Prediction for New Applicant:

Loan Approval: Yes

3. Smart Energy Consumption Monitoring

Libraries Used: Pandas, Bokeh

A. Using Pandas

Pandas is used to organize electricity consumption data in a DataFrame containing Appliance Name, Date, and Power Consumption. Using `groupby()`, electricity usage is analyzed by appliance and by date. The total power consumption is also calculated.

Application:

- Organize electricity usage data.
- Calculate appliance-wise power consumption.
- Analyze daily energy usage.
- Calculate total power consumption.

B. Data Preprocessing

Before analysis, the dataset is cleaned.

- Missing power values are replaced with the average value.
- Abnormal readings are removed.
- Daily and appliance-wise power consumption are calculated.

Benefits:

- Improves data accuracy.
- Removes incorrect values.
- Produces reliable energy reports.

C. Visualization using Bokeh

Bokeh is used to generate interactive visualizations.

- Bar Chart: Displays electricity consumption for each appliance.
- Line Chart: Shows daily electricity consumption trends.

These visualizations help identify high-energy appliances and monitor daily energy usage.

CODE:

```
import pandas as pd
import altair as alt

data = {
    "Appliance": ["AC", "Fan", "Light", "Refrigerator", "AC"],
    "Date": ["01-07", "01-07", "01-07", "01-07", "02-07"],
    "Power": [5.2, 1.0, None, 2.5, 6.0]
}

df = pd.DataFrame(data)

df["Power"] = df["Power"].fillna(df["Power"].mean())
df = df[df["Power"] < 10]

usage = df.groupby("Appliance")["Power"].sum().reset_index()
daily = df.groupby("Date")["Power"].sum().reset_index()

print("Power Usage by Appliance")
print(usage)

print("\nTotal Power Consumption =", df["Power"].sum())

bar = alt.Chart(usage).mark_bar().encode(
    x="Appliance:N",
    y="Power:Q",
    color="Appliance:N"
).properties(title="Power Usage by Appliance")

line = alt.Chart(daily).mark_line(point=True).encode(
    x="Date:N",
    y="Power:Q"
).properties(title="Daily Power Consumption")

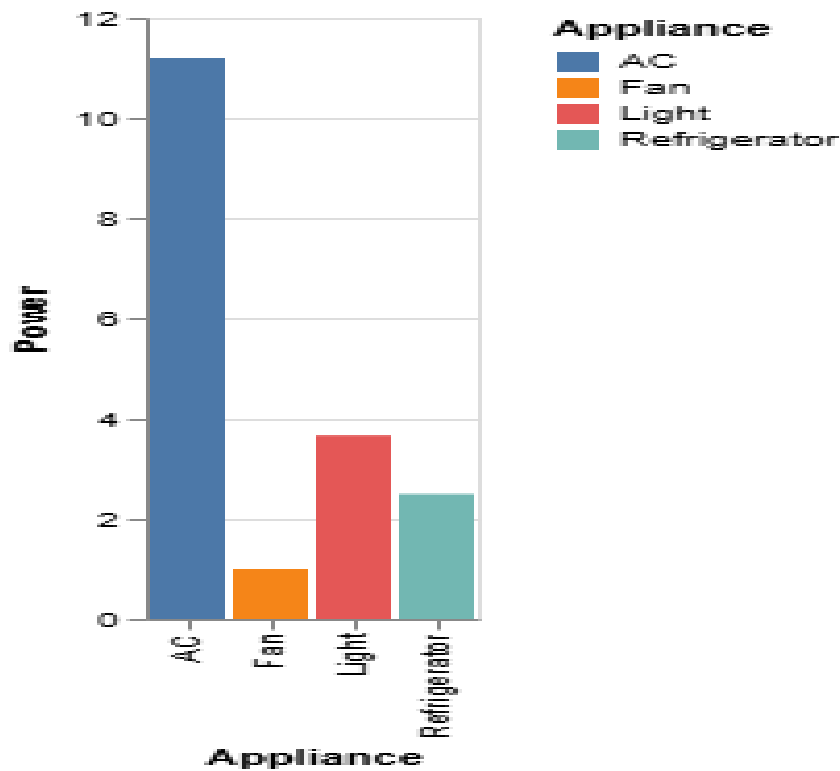
bar.show()
line.show()
```

OUTPUT:

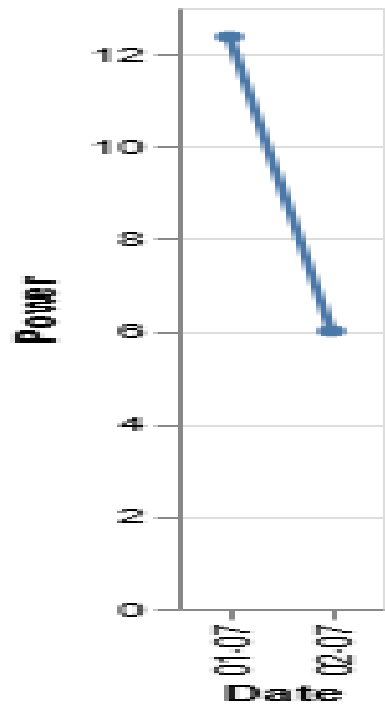
```
Power Usage by Appliance
  Appliance  Power
0        AC  11.200
1        Fan   1.000
2       Light   3.675
3 Refrigerator  2.500
```

```
Total Power Consumption = 18.375
```

Power Usage by Appliance



Daily Power Consumption



4. Online Learning Platform Performance Analysis

Libraries Used: Pandas, Altair

A. Using Pandas

Pandas is used to organize student learning data containing Student Name, Course Name, Time Spent, and Course Completion Percentage. The data is stored in a DataFrame and grouped course-wise to calculate the average completion percentage.

Application:

- Organize student learning records.
- Analyze course completion.
- Calculate average completion percentage.
- Compare student performance across courses.

B. Data Preprocessing

Before analysis, the dataset is cleaned.

- Missing study time values are replaced with the average time spent.
- Course-wise average completion percentage is calculated using `groupby()`.

These preprocessing steps improve the accuracy of performance analysis.

Benefits:

- Handles missing values.
- Improves data quality.
- Produces reliable learning reports.

C. Visualization using Altair

Altair is used to create interactive visualizations.

- Bar Chart: Displays the average course completion percentage for each course.

This visualization helps compare course performance and understand student learning progress.

CODE:

```
import pandas as pd
import altair as alt
```

```
data = {
    "Student": ["A", "B", "C", "D", "E"],
```

```

    "Course": ["Python", "Java", "Python", "C++", "Java"],
    "Time_Spent": [12, 8, None, 10, 15],
    "Completion": [90, 60, 80, 70, 95]
}

df = pd.DataFrame(data)

df["Time_Spent"] = df["Time_Spent"].fillna(df["Time_Spent"].mean())

course_avg = df.groupby("Course")["Completion"].mean().reset_index()

print("Student Data")
print(df)

print("\nAverage Completion by Course")
print(course_avg)

chart = alt.Chart(course_avg).mark_bar().encode(
    x="Course:N",
    y="Completion:Q",
    color="Course:N"
).properties(
    title="Average Course Completion"
)

chart.display()

```

OUTPUT:

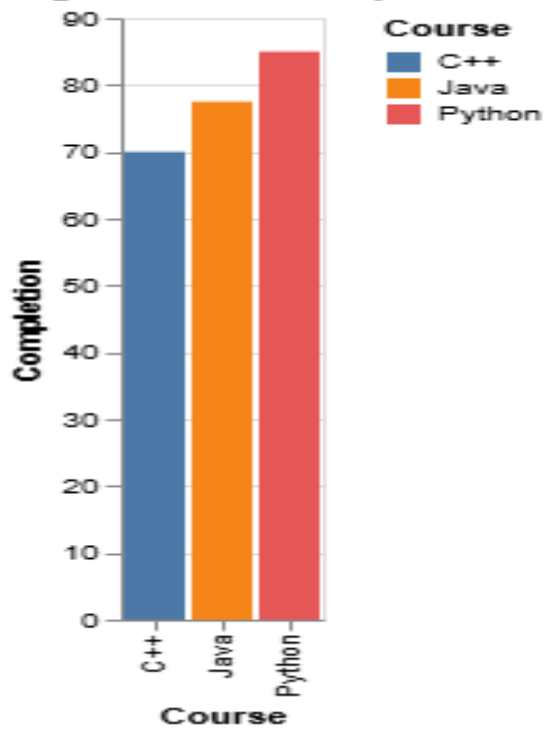
Student Data

	Student	Course	Time_Spent	Completion
0	A	Python	12.00	90
1	B	Java	8.00	60
2	C	Python	11.25	80
3	D	C++	10.00	70
4	E	Java	15.00	95

Average Completion by Course

	Course	Completion
0	C++	70.0
1	Java	77.5
2	Python	85.0

Average Course Completion



5. Public Transport Passenger Data Analysis

Libraries Used: Pandas, Plotly Express

A. Using Pandas

Pandas is used to organize passenger information in a DataFrame containing **Route Number, Passenger Count, Fare, and Date**. Using `groupby()`, passenger counts are analyzed route-wise and date-wise.

Application:

- Organize passenger records.
- Analyze passenger count by route.
- Analyze daily passenger count.
- Compare passenger traffic across routes.

B. Data Preprocessing

Before analysis, the passenger data is cleaned.

- Missing passenger counts are replaced with the average value.
- Passenger data is grouped by route and date.
- Clean data is prepared for visualization.

Benefits:

- Removes missing values.
- Improves analysis accuracy.
- Supports better transport planning.

C. Visualization using Plotly Express

Plotly Express is used to create interactive charts.

- Bar Chart: Displays passenger count for each route.
- Line Chart: Shows daily passenger count trends.

These visualizations help identify the busiest routes, monitor passenger flow, and support efficient public transport management.

CODE:

```
import pandas as pd
import plotly.express as px
```

```
data = {
    "Route": ["R1", "R2", "R1", "R3", "R2"],
```

```

    "Passengers": [120, 150, None, 100, 180],
    "Fare": [15, 20, 15, 10, 20],
    "Date": ["01-07", "01-07", "02-07", "02-07", "03-07"]
}

df = pd.DataFrame(data)

df["Passengers"] = df["Passengers"].fillna(df["Passengers"].mean())

route_data = df.groupby("Route")["Passengers"].sum().reset_index()
daily_data = df.groupby("Date")["Passengers"].sum().reset_index()

print("Passenger Data")
print(df)

print("\nPassengers by Route")
print(route_data)

print("\nDaily Passenger Count")
print(daily_data)

fig1 = px.bar(
    route_data,
    x="Route",
    y="Passengers",
    color="Route",
    title="Passengers by Route"
)

fig1.show()

fig2 = px.line(
    daily_data,
    x="Date",
    y="Passengers",
    markers=True,
    title="Daily Passenger Count"
)

fig2.show()

```

OUTPUT:

Passenger Data

	Route	Passengers	Fare	Date
0	R1	120.0	15	01-07
1	R2	150.0	20	01-07
2	R1	137.5	15	02-07

3 R3 100.0 10 02-07
4 R2 180.0 20 03-07

Passengers by Route

	Route	Passengers
0	R1	257.5
1	R2	330.0
2	R3	100.0

Daily Passenger Count

	Date	Passengers
0	01-07	270.0
1	02-07	237.5
2	03-07	180.0

